Tarea S1 · Adapta una variable y decide

Entrega: antes de que empiece S2 (miércoles 18 de noviembre, 10:00, hora de Perú) · repositorio personal con rama `tarea-s1` (o carpeta comprimida si no hay repo). Si no recuerdas cómo crear una rama y subirla: `git-y-ramas.md`. Se entrega con el formulario de Google del curso (el instructor da el enlace en clase); si tu repositorio es privado, añade al instructor como colaborador.

> **No confundas las dos cosas.** Esta tarea es un ejercicio de S1 sobre el caso **Online Retail II** (añadir una variable). Tu **caso propio, Bank Marketing**, es otro trabajo: se avanza de S1 a S5 y se entrega por avances. Léelo entero antes de S1 en «Tu caso: Bank Marketing». El avance 1 (RAW → MASTER → FS → ML y entrenamiento, como cuadernos) se presenta en S2 y se entrega a la misma hora que esta tarea; las fechas de los cuatro avances están en el apartado 9 de la hoja.

Qué se pide

Añadir **una variable nueva** a la capa FS del caso Online Retail II, reconstruir ML, reentrenar y **decidir con evidencia** si la variable se queda.

1. **Diseña** la variable. Debe usar solo datos anteriores al corte. Ejemplos: `monto_180d` (gasto en los 180 días previos), `ratio_devoluciones` (devoluciones/frecuencia), `dias_desde_ultima_devolucion`.
2. **Impleméntala** (ver «Dónde se toca» abajo) y documéntala en `fs_catalogo` (descripción, fórmula, unidad).
3. **Reconstruye** FS y ML y reentrena (`python -m ia_mlops.pipeline`).
4. **Compara dos ejecuciones** en MLflow (antes y después). Comprueba que la huella del dato cambió (los datos son otros) y que las demás condiciones son iguales.
5. **Decide** y escribe la decisión en `evidencias/plantilla-decision.md`: ¿se queda la variable? ¿por qué? Una decisión de «no» bien argumentada vale tanto como una de «sí».
6. **Entrega la traducción a `.py`** de los cuadernos que modificaste (`jupytext`) y la salida de las pruebas.

Dónde se toca (receta validada con `monto_180d`)

Seis lugares de `04-Proyecto-Base/` más el cuaderno de entrenamiento:

Archivo	Cambio
`sql/fs/01_crear_fs.sql`	fila en `fs_catalogo` y columna en `fs_cliente_corte`
`sql/fs/02_poblar_fs.sql`	cálculo en `por_cliente` (u otra CTE), columna en el `INSERT` y en el `SELECT`
`sql/ml/01_crear_ml.sql`	columna en `ml_dataset`
`sql/ml/02_poblar_ml.sql`	columna en el `INSERT` y en el `SELECT`

Archivo	Cambio
`ia_mlops/config.py`	nombre en `FEATURES`
`notebooks/04_entrenamiento_y_seguimiento.ipynb`	nombre en la lista `FEATURES` (si no, la prueba de equivalencia falla)

Regla de oro: todos los filtros usan `fecha_factura < :corte`. La prueba `pytest tests -k futuro` debe seguir en verde.

Resultado de referencia (no lo copies, compáralo)

Con la muestra, añadir `monto_180d` dio regresión logística valid 0,691 · test 0,739, frente a 0,698 · 0,747 sin ella: **no mejoró**; la decisión razonable era no conservarla. Tu variable puede comportarse distinto. Lo evaluado es el razonamiento, no que mejore.

Evidencia mínima

- Salida de `python -m pytest -q` (sin reglas rotas).
- Captura **o** exportación de la comparación de MLflow con ambas ejecuciones.
- `evidencias/plantilla-decision.md` completada.
- Scripts `.py` traducidos.

Rúbrica

Ver `rubrica.md`: 20 % reproducibilidad, 40 % técnica, 40 % análisis y decisión.

Honestidad

No se evalúa solo por capturas. Declara lo que no pudiste ejecutar y el error exacto. Citar la ayuda que recibiste (personas o herramientas).